

20 Agentic AI Interview Questions Most Mid-Level Candidates Fail

A brutally honest scorecard — three tiers of answer for every question. See what Reels-educated applicants say, what blog-skimmers say, and what a senior engineer actually wants to hear.

BY NISARG KADAM — LEAD AGENTIC AI ARCHITECT, SINGAPORE

AGENTIC AI BUILDER EXPERT BOOTCAMP, BATCH 4.0

Learn to build agents before agents learn to replace you.



Why I Built This

After interviewing hundreds of candidates for agentic AI roles, I've noticed three distinct tiers of answers for every question.

Tier 1 — The Reels-Educated Answer

Loud, confident, technically wrong. Picked up from a 30-second hype video with dubstep. Usually involves the words "bro," "literally," or "AGI."

Tier 2 — The Blog-Skimmer Answer

Surface-level correct. Safe. Generic. Will not survive a single follow-up question.

Tier 3 — What I Actually Want to Hear

Names the trade-offs. Names the failure modes. Names the decision criteria. **Hires itself.**

This document shows all three for 20 real questions. Use it as a scorecard if you're hiring. Use it as a mirror if you're interviewing. If your answer is closer to Tier 1 than Tier 3, the next 12 months of your career depend on closing that gap.

How To Read Every Question

Each slide has the same structure — a short, pointed question followed by three tiers of answer.

1

The Question

Short, pointed — the kind a senior engineer actually asks across a table.

2

Tier 1 / Tier 2

The hype-bro voice and the Wikipedia-energy answer. Brutally accurate counterexamples.

3

Tier 3

Trade-offs. Operational detail. Failure modes. Decision rules. The signal of someone who has shipped to production.

Workflow vs Agent in LangGraph

 **THE QUESTION:** What is the actual difference between a workflow and an agent in LangGraph?

 TIER 1 – REELS ANSWER ❌

"BRO. An agent is just an LLM with a vibe. A workflow is like if-else. Same thing honestly. I built 3 agents this weekend watching a YouTube short. Scale or die."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"An agent is an autonomous system that uses an LLM to make decisions, while a workflow is a sequence of predefined steps. LangGraph supports both through its StateGraph abstraction."

 TIER 3 – WHAT I WANT TO HEAR ✅

A workflow is a DAG where control flow is decided at design time — the developer hard-codes which node runs next. An agent is a graph where the LLM itself decides the next node at runtime via tool calls or routing logic. In LangGraph this shows up concretely: workflows use static `add_edge`, agents use `add_conditional_edges` where the routing function reads the LLM's last message to pick the next node. The trade-off is predictability vs flexibility — workflows are debuggable and cheap, agents handle open-ended tasks but cost more tokens and are harder to test. Pick workflow by default; upgrade to agent only when the decision space can't be enumerated upfront.

QUESTION 02

State Flow in a StateGraph

 **THE QUESTION:** How does state actually flow through a LangGraph StateGraph?

 TIER 1 – REELS ANSWER ❌

"It's literally just a dictionary bro. You pass it around. Super simple. LangGraph is just LangChain with extra steps fr fr."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"LangGraph uses a TypedDict or Pydantic model to represent state, which is passed between nodes. Each node can read and modify the state."

 TIER 3 – WHAT I WANT TO HEAR ✅

State is a typed schema (TypedDict, Pydantic, or dataclass) with reducers defined per field. Each node returns a partial state dict; LangGraph merges it into the global state using the reducer — default is overwrite, but for lists you use `operator.add` or `add_messages` to append. The critical nuance most candidates miss: reducers are what make parallel nodes safe. If two nodes update the same field without a reducer, you get a race condition. Also — state is checkpointed at every super-step, which is how you get human-in-the-loop interrupts and time-travel debugging for free.

QUESTION 03

What MCP Is For

 **THE QUESTION:** What is MCP and why should I care as an agent builder?

TIER 1 – REELS ANSWER ❌

"MCP is like the USB-C of AI bro. Literally Anthropic said that. It's gonna replace APIs. REST is dead. I saw it on a reel with dubstep."

TIER 2 – BLOG-SKIMMER ANSWER ❌

"Model Context Protocol is an open standard from Anthropic that lets LLMs connect to external tools and data sources in a standardised way."

TIER 3 – WHAT I WANT TO HEAR ✅

MCP is a protocol that decouples the tool provider from the tool consumer. Before MCP, every framework (LangChain, LlamaIndex, custom) re-implemented tool integrations – so a Jira integration had to be rewritten N times. MCP defines a server-client contract: servers expose tools, resources, and prompts; clients (any LLM host) discover and invoke them. The practical wins: write once, run in any MCP-compliant host; dynamic tool discovery at runtime; consistent auth and transport (stdio, SSE, HTTP). The trap – MCP doesn't solve agent orchestration, state, or reasoning. It solves the integration layer only. Treat it as plumbing, not as a framework replacement.

QUESTION 04

When NOT To Use an Agent

 **THE QUESTION:** When would you NOT use an agent and just use a deterministic chain?

TIER 1 – REELS ANSWER ❌

"Never bro. Agents are the future. If you're still using chains you're basically doing RPA in 2019. Agents AGI AGI AGI."

TIER 2 – BLOG-SKIMMER ANSWER ❌

"You might not need an agent for simple tasks where the steps are known in advance, like a basic question-answering system over a document."

TIER 3 – WHAT I WANT TO HEAR ✅

Use a chain or plain function when three conditions hold: (1) the steps are enumerable and stable, (2) cost and latency matter – agents burn 3-10x more tokens on reasoning, and (3) predictability is required for compliance or testing. A summarisation pipeline, a structured extraction job, a form-filler – all chains. Upgrade to an agent when the input space is genuinely open – "answer any customer query" – or when tool selection itself depends on intermediate results. A good heuristic: if you can draw the flowchart on a whiteboard without branches labelled "LLM decides", you don't need an agent.

Tool Calling vs Function Calling

 **THE QUESTION:** What's the difference between tool calling and function calling?

 TIER 1 – REELS ANSWER ❌

"Literally the same thing bro. OpenAI renamed it. Classic OpenAI move. Tool call = function call = API call = webhook. It's all vibes."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"Function calling was OpenAI's original term, tool calling is the newer generalised version. They mean roughly the same thing."

 TIER 3 – WHAT I WANT TO HEAR ✅

Historically, function calling was a single-function-per-turn API (OpenAI's 2023 release). Tool calling generalised it in three ways: multiple tools can be called in parallel in one turn, tools include non-function types (retrieval, code execution, computer use), and the schema supports `tool_choice` modes (auto, required, specific, none). The operational difference: with tool calling, you must handle a list of `tool_use` blocks per response and return matching `tool_result` blocks – getting the IDs wrong is the #1 bug. Also, parallel tool calls break naive sequential loops; you need to either execute concurrently or serialise with clear ordering guarantees.

Handling Tool Errors

 **THE QUESTION:** How do you handle tool call errors in an agent loop?

 TIER 1 – REELS ANSWER ❌

"Try-except bro. Wrap the whole thing. If it fails just retry 10 times. If still failing, add more prompt. 'You MUST not fail.' Works every time."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"You should use try-except blocks to catch exceptions from tool calls and pass the error back to the LLM so it can retry or choose a different approach."

 TIER 3 – WHAT I WANT TO HEAR ✅

Three layers. (1) Schema-level: validate tool args against the schema before execution – catch hallucinated fields early and return a structured error the LLM can reason about. (2) Execution-level: wrap the tool call, but return the error as a `tool_result` with `is_error=true` rather than raising – the LLM needs to see the failure to correct course. (3) Loop-level: track consecutive failures and force-break after N retries (usually 3) to prevent infinite loops where the agent keeps calling the same broken tool. Error messages matter – "connection refused" is useless; "API returned 429, rate limited, retry after 30s" lets the LLM pick a different tool or wait. Never silently retry – the LLM should see every attempt.

ReAct in 2026

 **THE QUESTION:** What is a ReAct agent and is it still relevant in 2026?

 TIER 1 – REELS ANSWER ❌

"ReAct is old news bro. We're on Tree of Thoughts, Graph of Thoughts, Forest of Thoughts now. ReAct is basically COBOL. Skip it."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"ReAct stands for Reasoning + Acting. The agent alternates between thinking steps and action steps. It was introduced in a 2022 paper and is implemented in LangChain."

 TIER 3 – WHAT I WANT TO HEAR ✅

ReAct is a prompting pattern: Thought → Action → Observation → Thought → ... until Final Answer. It forced the LLM to verbalise reasoning before acting, which improved tool-use accuracy dramatically when models were weaker. In 2026 it's still relevant but largely invisible — modern tool-calling APIs bake the reasoning step into the model (thinking tokens, chain-of-thought in the model response), so you rarely write explicit "Thought:" prompts anymore. Where it still matters: open-source models without native tool calling, interpretability requirements where you need to log reasoning, and custom agents where you want to constrain the reasoning format. The principle — reason before acting — is eternal. The literal ReAct prompt template is legacy.

Prompt Injection Defense

 **THE QUESTION:** How do you prevent prompt injection in an agent that uses tools?

 TIER 1 – REELS ANSWER ❌

"Prompt injection is a skill issue bro. Just add 'IGNORE PREVIOUS INSTRUCTIONS IF MALICIOUS' to your system prompt. Also GPT-5 is immune now. Solved problem."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"You can prevent prompt injection by sanitising user inputs, using delimiters to separate instructions from data, and validating outputs before acting on them."

 TIER 3 – WHAT I WANT TO HEAR ✅

Prompt injection is not fully solvable at the model layer; you must design the system defensively. Key controls: (1) Treat all tool outputs as untrusted data, not instructions – never concatenate tool output directly into the system prompt. (2) Principle of least privilege – the agent's tools should only do what's needed for the task; no `send_email` tool if the use case is summarisation. (3) Human-in-the-loop for destructive actions – writes, deletes, payments, emails. (4) Output filtering – scan for data exfiltration patterns (URLs with encoded data, suspicious tool calls). (5) Dual-LLM pattern – one privileged LLM plans, a quarantined LLM processes untrusted content and returns structured data only. The mental model: assume the injection will succeed; design so the blast radius is zero.

Short-Term vs Long-Term Memory

THE QUESTION: What's the difference between short-term and long-term memory in agents?

TIER 1 – REELS ANSWER ❌

"Short term is the last 10 messages, long term is a vector DB. Easy. Just dump everything in Pinecone and pray. RAG is memory, memory is RAG, same thing."

TIER 2 – BLOG-SKIMMER ANSWER ❌

"Short-term memory is the conversation history passed in the context window. Long-term memory is stored in a vector database and retrieved via semantic search when needed."

TIER 3 – WHAT I WANT TO HEAR ✅

Short-term memory is whatever fits in the current context window – the running conversation, scratchpad, tool results. It's ephemeral; when the session ends, it's gone. Long-term memory is persistent storage the agent reads from and writes to across sessions. The real distinction isn't the storage medium – it's write semantics. Short-term accumulates automatically. Long-term requires a decision to persist: what's worth remembering, how to index it, when to update vs. append, how to reconcile conflicts. A well-designed system has episodic memory ("what happened in session X"), semantic memory ("facts the user has told me"), procedural memory ("how this user prefers tasks done"). Vector DBs are one implementation; structured stores, knowledge graphs, and simple key-value are often better for semantic and procedural memory.

QUESTION 10

Human-in-the-Loop in LangGraph

 **THE QUESTION:** Explain how LangGraph handles human-in-the-loop.

TIER 1 – REELS ANSWER ❌

"You just add an if-statement bro. If confidence < 0.7, input(). Boom, HITL. I wrote a Medium article about it. 50k claps."

TIER 2 – BLOG-SKIMMER ANSWER ❌

"LangGraph supports human-in-the-loop by allowing you to pause the graph execution and wait for human input before continuing."

TIER 3 – WHAT I WANT TO HEAR ✔️

LangGraph uses checkpointers and interrupts. You configure a checkpointer (`MemorySaver`, `SqliteSaver`, `PostgresSaver`) which snapshots state at every super-step. Then either (a) set `interrupt_before` or `interrupt_after` on specific nodes – the graph halts, returns control, and you can inspect or edit state before resuming, or (b) call `interrupt()` inside a node to pause mid-execution and surface a question to the human. When the human responds, you resume with `Command(resume=...)` and the graph continues from the checkpoint. The power: you can edit state during the pause – approve a tool call, modify the plan, inject context – and time-travel to earlier checkpoints to retry with different inputs. The gotcha: without a durable checkpointer, pauses don't survive process restarts.

Why Multi-Agent Often Fails

 **THE QUESTION:** Why do multi-agent systems often perform worse than single agents?

 TIER 1 – REELS ANSWER ❌

"Because skill issue bro. You need 47 specialised agents, a CEO agent, a CFO agent, and a vibes agent. More agents = more intelligence. AGI is just enough agents."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"Multi-agent systems can have coordination overhead and communication failures between agents, leading to degraded performance compared to a single well-designed agent."

 TIER 3 – WHAT I WANT TO HEAR ✅

Three concrete failure modes. (1) Context fragmentation – each agent sees a slice of the problem, loses global context, and makes locally-optimal but globally-wrong decisions. (2) Error compounding – if each agent is 90% accurate, a 5-agent chain is $0.9^5 = 59\%$ end-to-end. (3) Coordination cost – agents negotiating, re-explaining, re-planning burns tokens and latency without adding signal. A single agent with good tools usually beats a multi-agent setup unless the task has genuinely parallel, independent subtasks (research N sources simultaneously) or requires strict role separation for safety (one agent writes, another reviews with a different system prompt). The honest rule: start with one agent and more tools. Split into multiple agents only when you can name the specific failure mode the split fixes.

Evaluating Agents in Production

 **THE QUESTION:** How would you evaluate an agent in production?

 **TIER 1 – REELS ANSWER** ❌
"You just look at the outputs bro. If they look good, they are good. Eyeball-driven development. I don't trust metrics, I trust my gut."

 **TIER 2 – BLOG-SKIMMER ANSWER** ❌
"You can evaluate agents using metrics like task completion rate, accuracy, and latency. Tools like LangSmith help track these over time."

 **TIER 3 – WHAT I WANT TO HEAR** ✅
Evaluation is layered. (1) Component-level: each tool call, each node output – is the tool being called with valid args? Does the retriever return relevant docs? (2) Trajectory-level: did the agent take a reasonable path, or did it loop, take unnecessary steps, use the wrong tool? Measured with trajectory eval datasets and LLM-as-judge. (3) Outcome-level: did the task succeed? Requires ground-truth labels or a judge model. (4) Production signals: latency percentiles, cost per task, tool failure rates, human escalation rate, user feedback. The trap most teams fall into: only measuring outcome – "it worked" – without instrumenting the trajectory. So when quality regresses you can't localise the cause. LangSmith, Langfuse, and custom OTEL traces are standard instrumentation. Run offline evals on a frozen dataset before every deploy.

QUESTION 13

When To Skip the Checkpointer

 **THE QUESTION:** What is a checkpointer and when would you skip it?

 TIER 1 – REELS ANSWER ❌

"Checkpointers are for noobs bro. Real agents don't forget. Just pass everything in the prompt. Context window go brrr."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"A checkpointer saves the state of the graph at each step, allowing you to resume from a previous state. You can skip it for simple, short-running tasks."

 TIER 3 – WHAT I WANT TO HEAR ✅

A checkpointer persists graph state per (thread_id, checkpoint_id). It enables: (a) resuming after crashes, (b) human-in-the-loop interrupts, (c) time-travel debugging – replay from any past checkpoint with modified state, (d) multi-turn conversations where the graph runs across multiple invocations. You skip it for stateless one-shot tasks where the whole graph runs in a single invocation and you don't need resumability – a summarisation endpoint, for example. You also swap to in-memory in unit tests for speed. What you should never skip: if the graph has any interrupt, any long-running node, or any user session spanning multiple requests – durable checkpointing is non-negotiable.

Retrieval vs Tool Use

 **THE QUESTION:** Explain the difference between retrieval and tool use in agentic systems.

 TIER 1 – REELS ANSWER ❌

"RAG is just a tool call bro. Everything is a tool call. Life is a tool call. Wake up sheeple."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"Retrieval means fetching documents from a knowledge base, while tool use means calling external functions or APIs. Both provide context to the LLM."

 TIER 3 – WHAT I WANT TO HEAR ✅

Retrieval is a specific kind of tool use where the tool is a search over a knowledge corpus, returning text chunks that get appended to context. Tool use is the broader category – any external capability the LLM can invoke: APIs, code execution, databases, other agents. The meaningful distinction is semantics: retrieval augments the LLM's knowledge; general tool use augments its capability. Operationally, retrieval is usually read-only, idempotent, and cheap – so it can be called liberally or even auto-injected before the LLM runs. Tool use often involves side effects, latency, auth, and cost – so it needs the LLM's explicit decision to call. Muddling them causes bad architectures: RAG systems that over-retrieve irrelevant docs, or agent systems that treat retrieval as a hard gate when the model already knows the answer.

QUESTION 15

Versioning and Deploying Agents



THE QUESTION: How do you version and deploy an agent?



TIER 1 – REELS ANSWER ❌

"Git push to main bro. YOLO deploy. If it breaks, prompt engineer it live in prod. That's agile."



TIER 2 – BLOG-SKIMMER ANSWER ❌

"You can version the prompts, model versions, and tool configurations. Deployment depends on your platform – often a container with an API endpoint."



TIER 3 – WHAT I WANT TO HEAR ✅

Four things must be versioned together because they form a single artifact: (1) model version and provider, (2) system prompts and prompt templates, (3) tool schemas and implementations, (4) graph topology. A change in any one can silently regress behaviour. Use a prompt registry (LangSmith, PromptHub, git) with semantic versions. Deploy artifacts as immutable bundles tied to a config hash. Run evals against a frozen dataset on every PR and gate merge on score. For rollout: shadow mode first – run new version in parallel, compare outputs, no user impact. Then canary 5% → 25% → 100% with automatic rollback on metric regression. Never edit prompts in production without a version bump. Never let the model provider auto-upgrade your model – pin versions.

QUESTION 16

The Role of Pydantic

 **THE QUESTION:** What's the role of Pydantic in an agentic stack?

 **TIER 1 – REELS ANSWER ❌**
"Pydantic is just a Python thing bro. Types are for Java devs. Just use dicts. Duck typing forever."

 **TIER 2 – BLOG-SKIMMER ANSWER ❌**
"Pydantic is used to define data models for structured inputs and outputs, including tool schemas and state in LangGraph."

 **TIER 3 – WHAT I WANT TO HEAR ✔️**
Pydantic plays four roles. (1) Tool schemas – Pydantic models compile to JSON Schema, which is what tool-calling APIs consume. One source of truth beats hand-written JSON schemas that drift from the code. (2) Structured outputs – enforce the LLM to return valid JSON matching a model, with automatic validation and retry on failure. (3) State definition – LangGraph supports Pydantic state, giving you validation at every node boundary. (4) Config – environment config, agent config, model params – all validated at startup, failing loud instead of deep in production. The upgrade from v1 to v2 matters – v2 is 5-50x faster and has stricter validation. Mid-level candidates who still write v1 syntax (`.dict()`, `@validator`) signal stale practice.

QUESTION 17

Streaming in Agents

 **THE QUESTION:** How do streaming responses work in an agent, and why do they matter?

 TIER 1 – REELS ANSWER ❌

"Streaming is just yield bro. Python generator. Easy. It's just vibes moving faster."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"Streaming sends the LLM's response token by token as it's generated, improving perceived latency. LangGraph supports streaming via the stream method."

 TIER 3 – WHAT I WANT TO HEAR ✅

There are three orthogonal streams in an agentic system, and mid-level candidates usually conflate them. (1) Token stream – the raw LLM output, token by token, for UI responsiveness. (2) Event stream – node-level events from the graph: "node X started", "tool Y called with args Z", "state updated". This is what powers agent trace UIs. (3) State stream – after each super-step, the full updated state is emitted; consumers can diff to detect changes. LangGraph exposes all three via `stream_mode='values' | 'updates' | 'messages' | 'custom'`. Why it matters: for a chatbot you want token + message streaming; for an agent dashboard you want event + update streaming; for debugging you want all of them. Picking the wrong mode means either broken UX or massive overhead piping data the UI doesn't need.

Fine-Tuning vs Building an Agent



THE QUESTION: What's the difference between fine-tuning and building an agent for a domain-specific task?



TIER 1 – REELS ANSWER ❌

"Fine-tuning is dead bro. Agents replaced it. Why train when you can prompt. GPT-5 fine-tuning is a scam by OpenAI to sell compute."



TIER 2 – BLOG-SKIMMER ANSWER ❌

"Fine-tuning adjusts the model's weights for a specific task, while an agent uses a general model with prompts and tools. Agents are faster to build but fine-tuning can give better accuracy on narrow tasks."



TIER 3 – WHAT I WANT TO HEAR ✔️

They solve different problems. Fine-tuning changes what the model knows or how it behaves – tone, format, domain jargon, implicit knowledge. Agent-building changes what the model can do – access to tools, memory, and real-time data. You fine-tune when: you have stable, narrow behaviour you want baked in (e.g., extract medical entities in a specific ontology); the cost-per-call of a large model is prohibitive and a smaller fine-tuned model suffices; or you need offline/air-gapped deployment. You build an agent when the task requires fresh data, external systems, or decision-making over dynamic inputs. The pragmatic sequence: prompt engineer first, add tools/retrieval second, build a full agent third, fine-tune last – and only when you have thousands of labelled examples and a clear evaluation set showing prompting plateaued.

Managing Cost in Agentic Systems

 **THE QUESTION:** How do you handle cost in an agentic system?

 TIER 1 – REELS ANSWER ❌

"You don't bro. Raise a Series A. Burn VC money. Agents are the future, costs will come down, scale now worry later."

 TIER 2 – BLOG-SKIMMER ANSWER ❌

"You can reduce costs by using smaller models, caching responses, and limiting the number of tool calls. Monitoring with LangSmith helps track spend."

 TIER 3 – WHAT I WANT TO HEAR ✅

Cost in agents is dominated by token consumption, which scales with (context size × tool-calling loop iterations × model tier). Levers in order of impact: (1) Model routing – use a cheap model (Haiku, Gemini Flash) for classification, routing, and simple tool calls; reserve Opus/Sonnet for hard reasoning. A supervisor pattern where a cheap model delegates to an expensive one can cut cost 5-10x. (2) Context hygiene – prune tool results aggressively; don't carry full retrieval chunks across turns; summarise old conversation. (3) Caching – prompt caching (Anthropic), semantic caching for repeat queries, tool-result caching for idempotent calls. (4) Loop bounds – hard cap on iterations, tool calls per turn, tokens per session. (5) Batching – combine parallel tool calls where the API supports it. Instrument cost per task from day one; debugging cost regressions without traces is hopeless.

Deciding Whether To Build an Agent At All

THE QUESTION: If I gave you a new business problem today, how would you decide whether to build an agent for it?

❌ TIER 1 – REELS ANSWER ❌

"Every problem is an agent problem bro. That's the whole point. I build agents for literally everything. My coffee machine has 3 agents. Agents agents agents."

⚠️ TIER 2 – BLOG-SKIMMER ANSWER ❌

"You should evaluate if the problem requires multi-step reasoning, access to external tools, or dynamic decision-making. If yes, an agent may be appropriate."

✅ TIER 3 – WHAT I WANT TO HEAR ✅

Run it through four filters before writing any code. (1) Is the task genuinely open-ended? If the full decision tree fits on a whiteboard, it's a workflow, not an agent. (2) Does it need external systems the LLM can't reach natively? If no tools are required and it's pure text transformation, it's a prompt, not an agent. (3) What's the cost of a wrong answer? High-stakes, low-reversibility tasks (financial, medical, legal, destructive ops) need human-in-the-loop – which means the "agent" is really a copilot, and scoping it that way changes the design. (4) Can you evaluate it? If you can't define success precisely enough to measure it, you can't improve it – and agents silently degrade without eval. If all four pass, build the agent. If not, build the simpler thing and revisit in 3 months. The failure mode I see most often: building an agent for a problem that was actually a rules engine with a nice UI.

The Pattern Across All 20 Questions

Notice the pattern that repeats across every single question:

Tier 1 is Confident

Because it doesn't know what it doesn't know. Loud certainty is the loudest signal of ignorance.

Tier 2 is Safe

Because it hasn't shipped anything. Wikipedia energy. Correct-sounding. Survives zero follow-up questions.

Tier 3 is Specific

Because it's been paged at 3 AM when the thing broke. Specificity is the fingerprint of production experience.

The gap between Tier 2 and Tier 3 is the most valuable gap in your career right now. Everyone can reach Tier 2 by reading docs. **Only builders reach Tier 3.**



The Uncomfortable Takeaway

If You're Hiring

These 20 questions will separate the reel-watchers from the builders in under 30 minutes. Most resumes don't. Use this as your scorecard — the depth of the answer tells you everything about whether someone has shipped or just read.

If You're a Candidate

The answer key above is not a script to memorise. The depth comes from shipping — from the first agent that loops forever in production, from the retrieval system that returns garbage, from the tool call that silently fails. You can't fake Tier 3. You have to earn it.

The Bootcamp Exists Because

You can't shortcut the shipping. But you can shortcut the scar tissue — by learning from someone who has already bled for these answers.

Next Steps — Agentic AI Builder Expert Bootcamp

Batch 6.0 — Weekend format. 10 sessions, 2 hours each. Real production builds, not notebook demos.

Reach Out

 nisargkadam23@gmail.com

 +65 85970451 | +91 9730860905

 linkedin.com/in/nisargkadam

Nisarg Kadam

Lead Agentic AI Architect, Singapore

4x Global Hackathon Champion | International Keynote Speaker

Learn to build agents before agents learn to replace you.